



Ariel Soto

Escuela de Administración y Negocios
Universidad de Concepción · Campus Chillán

SEMESTRE 1 · 2026



Fundamentos de Programación para Análisis Económico

MÓDULO II · SEMANA 6 · SESIÓN 1

Pivoteo de Datos

Dejar los datos ordenados

La **Semana 5** trabajó datos ya ordenados. Pero los datos **reales** casi nunca llegan así.

La idea rectora de toda la semana — **tidy data**:

Una tabla está ordenada cuando **cada variable es una columna y cada observación es una fila**. Casi toda la limpieza consiste en llevar datos a esta forma.

Al terminar esta sesión podrás:

- ▶ Distinguir formato **wide** (ancho) de **long** (largo)
- ▶ Transformar entre ambos con `pivot_longer()` y `pivot_wider()`
- ▶ Separar y unir columnas con `separate()` y `unite()`

Wide vs long: la misma información, dos formas

Wide (ancho) — un año por columna

```
  region a2020 a2021 a2022
1  Ñuble 380000 410000 450000
2  Biobío 520000 560000 610000
3   Maule 400000 430000 470000
4 Araucanía 360000 375000 400000
```

Cómodo de leer, como en Excel. Pero “año” es una variable **escondida en los nombres** de las columnas → no es tidy.

Long (largo) — una fila por región-año

```
# A tibble: 6 × 3
  region anio  ingreso
  <chr> <chr>   <int>
1 Ñuble  2020    380000
2 Ñuble  2021    410000
3 Ñuble  2022    450000
4 Biobío 2020    520000
5 Biobío 2021    560000
6 Biobío 2022    610000
```

Lo que dplyr y los gráficos **necesitan**.

Wide vs long: la misma información, dos formas

“Long” format

country	year	metric
x	1960	10
x	1970	13
x	2010	15
y	1960	20
y	1970	23
y	2010	25
z	1960	30
z	1970	33
z	2010	35

“Wide” format

country	yr1960	yr1970	yr2010
x	10	13	15
y	20	23	25
z	30	33	35

`pivot_longer()` — **wide** → **long**



De ancho a largo

```
ingresos_wide |>
  pivot_longer(
    cols      = c(a2020, a2021, a2022), # columnas a "apilar"
    names_to  = "anio",                 # nueva columna de etiquetas
    values_to = "ingreso"               # nueva columna de valores
  )
```

```
# A tibble: 12 × 3
  region    anio ingreso
  <chr>    <chr>   <int>
1 Ñuble   a2020   380000
2 Ñuble   a2021   410000
3 Ñuble   a2022   450000
4 Biobío  a2020   520000
5 Biobío  a2021   560000
6 Biobío  a2022   610000
7 Maule   a2020   400000
8 Maule   a2021   430000
9 Maule   a2022   470000
10 Araucanía a2020  360000
11 Araucanía a2021  375000
```

Limpiar nombres al pivotar

```
ingresos_long <- ingresos_wide |>
  pivot_longer(cols = starts_with("a"),
               names_to = "anio", values_to = "ingreso",
               names_prefix = "a") |>      # quita la "a": a2020 → 2020
  mutate(anio = as.numeric(anio))        # y a número (Semana 3)
ingresos_long
```

```
# A tibble: 12 × 3
  region    anio ingreso
  <chr>    <dbl>   <int>
1 Ñuble    2020   380000
2 Ñuble    2021   410000
3 Ñuble    2022   450000
4 Biobío   2020   520000
5 Biobío   2021   560000
6 Biobío   2022   610000
7 Maule    2020   400000
8 Maule    2021   430000
9 Maule    2022   470000
10 Araucanía 2020   360000
11 Araucanía 2021   375000
```

Por qué el formato largo gana

Recién en formato largo se puede usar dplyr de forma natural:

```
ingresos_long |>
  group_by(anio) |>
  summarise(ingreso_medio = mean(ingreso))
```

```
# A tibble: 3 × 2
  anio ingreso_medio
  <dbl>         <dbl>
1  2020         415000
2  2021         443750
3  2022         482500
```

Esto sería **imposible de hacer limpio** en formato wide. Pivotar a largo es el paso previo a casi todo análisis y gráfico.

`pivot_wider()` — long → wide



De largo a ancho: la operación inversa

Útil para **presentar** resultados (tablas para un informe):

```
ingresos_long |>
  pivot_wider(names_from = anio,      # qué columna se vuelve encabezados
              values_from = ingreso) # qué columna llena las celdas
```

```
# A tibble: 4 × 4
  region    `2020` `2021` `2022`
  <chr>    <int>  <int>  <int>
1 Ñuble    380000  410000  450000
2 Biobío    520000  560000  610000
3 Maule     400000  430000  470000
4 Araucanía 360000  375000  400000
```

Se analiza en largo, se presenta en ancho. `pivot_wider` suele ser el último paso, no el primero.

Caso típico: tabla de resultados

Combinar `group_by` (S5) + `pivot_wider` arma una tabla región × año lista para informe:

```
ingresos_long |>
  group_by(region, anio) |>
  summarise(ingreso = mean(ingreso), .groups = "drop") |>
  pivot_wider(names_from = anio, values_from = ingreso)
```

```
# A tibble: 4 × 4
  region    `2020` `2021` `2022`
  <chr>    <dbl>  <dbl>  <dbl>
1 Araucanía 360000  375000  400000
2 Biobío    520000  560000  610000
3 Maule     400000  430000  470000
4 Ñuble     380000  410000  450000
```

El patrón **resumir** → **ensanchar** produce las tablas de los informes económicos. Anticipa el Módulo III (comunicación).

wide ↔ long son inversas

`pivot_longer` y `pivot_wider` se deshacen mutuamente.

Regla mnemotécnica:

longER = más filas · widER = más columnas

Ejemplo: pivotar habilita el análisis temporal

```
ingresos_wide |>
  pivot_longer(starts_with("a"), names_to = "anio",
               values_to = "ingreso", names_prefix = "a") |>
  mutate(anio = as.numeric(anio)) |>
  group_by(region) |>
  summarise(crecimiento = last(ingreso) / first(ingreso) - 1)
```

```
# A tibble: 4 × 2
  region      crecimiento
  <chr>         <dbl>
1 Araucanía    0.111
2 Biobío       0.173
3 Maule        0.175
4 Ñuble        0.184
```

El código produce el número (crecimiento del ingreso por región 2020→2022); **el economista lo interpreta:** ¿qué región creció más?

`separate()` y `unite()`



Una columna con varios datos adentro

Problema real: una columna "Agricultura_Ñuble" mezcla dos variables.

```
datos <- data.frame(grupo = c("Agricultura_Ñuble", "Servicios_Biobío"),  
                    ingreso = c(450000, 920000))  
  
datos |> separate(grupo, into = c("sector", "region"), sep = "_")
```

	sector	region	ingreso
1	Agricultura	Ñuble	450000
2	Servicios	Biobío	920000

`unite()` hace lo inverso (juntar columnas en una):

```
datos |> separate(grupo, into = c("sector","region"), sep = "_") |>  
  unite("grupo", sector, region, sep = "-")
```

	grupo	ingreso
1	Agricultura-Ñuble	450000
2	Servicios-Biobío	920000



Cierre: lo que nos llevamos hoy

Función	Hace
<code>pivot_longer()</code>	wide → long (más filas)
<code>pivot_wider()</code>	long → wide (más columnas)
<code>separate()</code>	parte una columna en varias
<code>unite()</code>	junta varias columnas en una

Todo al servicio de **tidy data**. `separate()` está superada por `separate_wider_delim()` en `tidyr` reciente, pero sigue funcionando y es más simple para empezar.

Puente a la Sesión 2: ya damos forma a la tabla; ahora enfrentamos lo que falta **dentro** de ella — los **datos faltantes** y los **outliers**.



Sesión 1 — Fin

Break de 10–15 minutos

A continuación: **Sesión 2 — Datos Faltantes y Outliers** `NA` ·

`is.na()` · eliminar vs imputar · IQR

